

Университет МГУ-ППИ в Шэньчжэне

Выпускная квалификационная работа бакалавра

Исследование методов обучения
с подкреплением для пошаговых
игр со случайностью

Автор: Дай Ганьилэ

Научный руководитель: Оганесян Павел Артурович

Направление: Математика и прикладная математика

Университет: Университет МГУ-ППИ в Шэньчжэне

Шэньчжэнь, 2026

Аннотация

Настоящая работа посвящена исследованию методов обучения с подкреплением для пошаговых игр, в которых агент принимает решения последовательно, а результат партии зависит как от стратегии игроков, так и от особенностей игровой среды. В качестве первого проверочного примера рассматривается игра Tic-Tac-Toe, которая является простой детерминированной игрой с полной информацией и может быть полностью описана алгоритмически. На этом примере анализируется влияние функции награды на качество обученной стратегии: сравниваются варианты награды только за победу и награды за победу и ничью.

В качестве основной более сложной задачи рассматривается игра Einstein Würfelt Nicht!, где присутствуют случайный бросок кубика, изменяемая начальная конфигурация поля и более сложная структура допустимых ходов. Для этой игры строится среда обучения, задаются представление состояния, пространство действий и маска допустимых действий. Обучение агента проводится с использованием метода PPO в многоагентной постановке. Результаты оцениваются в играх против алгоритмического противника и по статистическим показателям качества, таким как доля побед, доля поражений, доля ничьих и не проигранных партий.

Проведённые эксперименты показывают, что для простых полностью определённых игр точные алгоритмические методы обладают преимуществом по надёжности и интерпретируемости. Однако при увеличении числа состояний, наличии случайности и сложной структуры ветвления методы обучения с подкреплением становятся более перспективным инструментом построения игровой стратегии.

Ключевые слова: обучение с подкреплением; пошаговые игры; многоагентное обучение; PPO; Gymnasium; RLlib; Tic-Tac-Toe; Einstein Würfelt Nicht!.

Contents

Аннотация	1
1 Введение	4
1.1 Цель работы	5
1.2 Задачи и основные результаты работы	5
1.3 Структура работы	5
2 Основы обучения с подкреплением для пошаговых игр	6
2.1 Основные понятия обучения с подкреплением	6
2.2 Марковская постановка задачи	6
2.3 Многоагентная постановка	7
2.4 Маска допустимых действий	7
2.5 Алгоритм PPO	8
3 Используемые инструменты и программная реализация	10
3.1 Среда разработки и экспериментальный фреймворк	10
3.2 Многоагентная архитектура с общей политикой	10
3.3 Структура наблюдения и ограничения действий	11
3.4 Рабочий процесс обучения на базе RLlib	11
4 Экспериментальное исследование на примере Tic-Tac-Toe	12
4.1 Описание игры	12
4.2 Состояние и действия	12
4.3 Вариант награды только за победу	13
4.4 Вариант награды за победу и ничью	14
4.5 Метрики качества	14
4.6 Постановка эксперимента	15
4.6.1 Смешанный пул противников	16
4.6.2 Асимметричная выборка ролей	16
4.7 Результаты экспериментов	16
4.8 Вывод по эксперименту Tic-Tac-Toe	23
5 Обучение агента для игры Einstein Würfelt Nicht!	24
5.1 Правила игры	24

5.2	Особенности задачи	25
5.3	Представление состояния	25
5.4	Функция награды	26
5.5	Пространство действий	26
5.6	Модель и архитектура	27
5.7	Алгоритм обучения	28
5.8	Данные обучения и вычислительные условия	28
5.9	Результаты против базового алгоритмического противника .	29
5.10	Анализ результатов	31
6	Заключение	32

1 Введение

Обучение с подкреплением является одним из направлений машинного обучения, в котором агент формирует стратегию поведения на основе взаимодействия со средой. В отличие от обучения с учителем, где для каждого входного объекта заранее известен правильный ответ, в обучении с подкреплением агент получает информацию о качестве своих действий через награду. Такая постановка естественно возникает в задачах управления, планирования, робототехники и игровых моделях [1].

Пошаговые игры представляют собой удобный класс задач для применения и анализа методов обучения с подкреплением. В таких играх процесс принятия решений дискретен: на каждом шаге игрок наблюдает состояние, выбирает одно из допустимых действий и получает новый игровой статус. Если игра содержит двух или более игроков, то задача становится многоагентной, поскольку результат действия одного игрока зависит от поведения другого игрока. Дополнительную сложность создаёт случайность, например бросок кубика или случайная начальная конфигурация.

Случайность принципиально усложняет анализ игры. В детерминированной игре дерево вариантов содержит только узлы выбора игроков, тогда как в игре с броском кубика появляются chance-узлы, соответствующие случайным исходам. При увеличении глубины партии число ветвей быстро растёт, и прямой перебор всех вариантов становится менее удобным. Поэтому для игр со случайными событиями представляет интерес model-free подход: стратегия обучается на большом числе сыгранных партий, не требуя явного полного перебора дерева игры.

В данной работе рассматриваются две игры. Первая игра, Tic-Tac-Toe, используется как простой проверочный пример. Она имеет малое число состояний, полную информацию и может быть решена точными алгоритмическими методами. Поэтому её основная роль в работе состоит не в демонстрации превосходства обучения с подкреплением, а в проверке корректности игровой среды, функции награды и процедуры обучения.

Вторая игра, Einstein Würfelt Nicht!, является более содержательным объектом исследования. В этой игре присутствуют случайный бросок кубика, выбор допустимых фигур по значению кубика и изменяемое начальное расположение фигур. Эти особенности увеличивают число

возможных игровых ситуаций и усложняют построение универсальной стратегии вручную.

1.1 Цель работы

Целью работы является построение и экспериментальное исследование моделей обучения с подкреплением для пошаговых игр со случайностью на примере Tic-Tac-Toe и Einstein Würfelt Nicht!.

1.2 Задачи и основные результаты работы

В рамках поставленной цели в работе решаются следующие содержательные задачи:

1. построить многоагентные среды для Tic-Tac-Toe и Einstein Würfelt Nicht! с поддержкой масок допустимых действий;
2. исследовать в Tic-Tac-Toe влияние функции награды, смешанного пула противников, исторического пула стратегий и разделения каналов признаков;
3. разработать для Einstein Würfelt Nicht! среду обучения, учитывающую случайный бросок кубика, начальную расстановку фигур и reward shaping;
4. обучить стратегии методом PPO и оценить их против случайных, minimax или других базовых алгоритмических противников.

1.3 Структура работы

Дальнейшая структура работы такова. Во втором разделе излагаются основы обучения с подкреплением для пошаговых игр. В третьем разделе описываются используемые библиотеки и программная реализация. В четвёртом разделе рассматривается экспериментальная проверка на игре Tic-Tac-Toe. В пятом разделе строится и исследуется модель для игры Einstein Würfelt Nicht!. В заключении формулируются основные выводы и направления дальнейшего развития.

2 Основы обучения с подкреплением для пошаговых игр

2.1 Основные понятия обучения с подкреплением

В обучении с подкреплением агент взаимодействует со средой во времени. На шаге t среда находится в состоянии s_t , агент выбирает действие a_t , после чего получает награду r_t и переходит в новое состояние s_{t+1} . Стратегия агента обозначается через π и задаёт правило выбора действия по наблюдаемому состоянию:

$$a_t \sim \pi(\cdot \mid s_t). \quad (2.1)$$

Цель агента состоит в максимизации ожидаемой суммарной награды. В простейшем случае дисконтированная суммарная награда имеет вид

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad (2.2)$$

где $\gamma \in [0, 1]$ — коэффициент дисконтирования.

Для пошаговой игры состояние s_t обычно содержит положение фигур на поле, текущего игрока, дополнительные случайные параметры и другую информацию, необходимую для выбора действия. Действие a_t соответствует одному допустимому ходу.

2.2 Марковская постановка задачи

Многие задачи обучения с подкреплением формализуются как марковский процесс принятия решений. Такая модель задаётся набором

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad (2.3)$$

где $\mathcal{S}$ — множество состояний, $\mathcal{A}$ — множество действий, P — вероятности переходов, R — функция награды, а γ — коэффициент дисконтирования.

В играх со случайностью переход из состояния s_t в состояние s_{t+1} может зависеть не только от выбранного действия, но и от случайного

события. В игре Einstein Würfelt Nicht! таким случайным событием является бросок кубика.

Определение 2.1. Двухигровая нулевая марковская игра задаётся набором

$$\mathcal{G} = (\mathcal{S}, \mathcal{A}_1, \mathcal{A}_2, P, R_1, R_2, \gamma), \quad (2.4)$$

где $\mathcal{S}$ — множество состояний, $\mathcal{A}_1$ и $\mathcal{A}_2$ — пространства действий двух игроков, $P(s' \mid s, a_1, a_2)$ — вероятности переходов, а R_1 и R_2 — функции награды игроков. В нулевой сумме выполняется условие

$$R_1(s, a_1, a_2) = -R_2(s, a_1, a_2). \quad (2.5)$$

В пошаговой игре на каждом ходе активен только один игрок, поэтому совместное действие можно рассматривать как действие текущего игрока при фиксированном ожидании второго игрока. Такая формализация удобна для описания Tic-Tac-Toe и Einstein Würfelt Nicht! как многоагентных сред.

2.3 Многоагентная постановка

Если в игре участвуют два игрока, то среда может рассматриваться как многоагентная. Каждый игрок имеет свою очередь хода и выбирает действие в соответствии со своей стратегией. В данной работе используется подход с общей политикой для двух игроков: одна и та же обучаемая модель применяется к обоим игрокам, а различие между ними кодируется в состоянии среды.

Такой подход уменьшает число обучаемых параметров и позволяет использовать симметрию игры. Однако он требует аккуратного представления состояния, чтобы модель могла корректно различать ситуацию текущего игрока и ситуацию противника.

2.4 Маска допустимых действий

Во многих играх не все действия допустимы в каждом состоянии. Например, в Tic-Tac-Toe нельзя поставить знак в уже занятую клетку. В игре Einstein Würfelt Nicht! допустимость хода зависит от значения кубика, положения фигур и правил движения.

Для учёта этого ограничения используется маска допустимых действий:

$$m(a \mid s) = \begin{cases} 1, & \text{если действие } a \text{ допустимо в состоянии } s, \\ 0, & \text{иначе.} \end{cases} \quad (2.6)$$

Маска позволяет исключить невозможные ходы из выбора действия и тем самым улучшает стабильность обучения. В реализации маска применяется на уровне логитов политики. Если $z_\theta(s, a)$ — исходный логит действия a , то после маскирования используется величина

$$z'_\theta(s, a) = z_\theta(s, a) + \begin{cases} 0, & a \in \mathcal{A}_{\text{legal}}(s), \\ -10^9, & a \notin \mathcal{A}_{\text{legal}}(s). \end{cases} \quad (2.7)$$

После применения softmax

$$\pi_\theta(a \mid s) = \frac{\exp(z'_\theta(s, a))}{\sum_{a'} \exp(z'_\theta(s, a'))} \quad (2.8)$$

слагаемое для недопустимого действия содержит множитель $\exp(-10^9)$ и в численной реализации становится равным нулю с точностью машинной арифметики. Такая схема не меняет пространство действий среды, но предотвращает расходование выборки на заведомо невозможные ходы и исключает выбор действия, ведущего к недопустимому переходу состояния.

2.5 Алгоритм PPO

В работе используется алгоритм PPO (Proximal Policy Optimization), относящийся к классу policy gradient методов [2]. Общая идея PPO состоит в том, чтобы обновлять стратегию не слишком резко, ограничивая отличие новой политики от старой. Это повышает устойчивость обучения по сравнению с прямым градиентным обновлением политики.

Базовая часть целевой функции PPO называется clipped surrogate objective и записывается в виде

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (2.9)$$

где $r_t(\theta)$ — отношение вероятностей действия при новой и старой политике, $\hat{A}_t$ — оценка преимущества, а ε — параметр ограничения обновления.

В используемой actor-critic архитектуре эта величина является только policy-частью обновления. Важно различать математическую целевую функцию, которую PPO максимизирует, и loss-функцию, которую библиотека минимизирует при обучении нейронной сети. Полная максимизируемая цель включает ошибку value-функции и энтропийный бонус:

$$J^{\text{PPO}}(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 \mathbb{E}_t [S(\pi_\theta(\cdot | s_t))]. \quad (2.10)$$

Соответствующая минимизируемая loss-функция имеет противоположные знаки:

$$\mathcal{L}^{\text{PPO}}(\theta) = -L^{\text{CLIP}}(\theta) + c_1 L^{\text{VF}}(\theta) - c_2 \mathbb{E}_t [S(\pi_\theta(\cdot | s_t))]. \quad (2.11)$$

Здесь $L^{\text{VF}}(\theta)$ — loss value-функции, обычно задаваемый как среднеквадратичная ошибка прогноза ценности состояния, $S(\pi_\theta)$ — энтропия политики, а c_1 и c_2 — коэффициенты value-loss и энтропии соответственно. В текущей реализации c_1 соответствует параметру `RLlib vf_loss_coeff`; явно он не переопределялся, поэтому использовалось значение по умолчанию 1.0. Энтропийный член поощряет исследование действий и снижает риск преждевременного схлопывания политики. В программной реализации эта идея соответствует наличию двух выходных ветвей модели: policy-head для логитов действий и value-head для оценки состояния. Поэтому в логах обучения отдельно отображаются policy loss, value loss и entropy.

В экспериментах Tic-Tac-Toe коэффициент энтропии задаётся расписанием $0.001 \rightarrow 0.0002 \rightarrow 0$, что усиливает исследование в начале обучения и постепенно снижает случайность политики. В эксперименте Einstein Würfelt Nicht! используется фиксированное значение $c_2 = 0.01$.

3 Используемые инструменты и программная реализация

3.1 Среда разработки и экспериментальный фреймворк

Эксперименты реализованы на Python; построение и обучение нейронных сетей выполняется с использованием PyTorch, пространства наблюдений и действий описываются в формате Gymnasium, а многоагентное обучение PPO запускается средствами Ray RLlib [5]. На каждом шаге среда активирует только текущего игрока и возвращает ему наблюдение, маску допустимых действий и награду; завершение партии задаётся общим признаком окончания episode.

3.2 Многоагентная архитектура с общей политикой

В экспериментах используется shared-policy схема, при которой оба игрока отображаются на одну обучаемую политику. Такая настройка удобна для симметричных двухигровых игр с нулевой суммой и позволяет обучать модель на опыте ходов обеих сторон. Обучаемая стратегия задаётся actor-critic сетью: policy-head формирует logits действий, а value-head оценивает ценность состояния.

Чтобы одна и та же политика могла применяться к разным сторонам игры, наблюдения приводятся к перспективе текущего игрока. В Tic-Tac-Toe это реализуется через относительное кодирование: собственные знаки текущего игрока получают код 1, а знаки противника — код 2 независимо от того, является текущий игрок первым или вторым. В Einstein Würfelt Nicht! дополнительно используется геометрическое преобразование: наблюдение второго игрока поворачивается на 180° , а знаки фигур меняются на противоположные. В результате обе стороны видят задачу в единой системе координат, что позволяет использовать одну нейронную сеть для action selection в обеих ролях.

Формально, если исходная доска задана матрицей $\mathbf{B} \in \mathbb{Z}^{N \times N}$, то для

второго игрока используется преобразование

$$[\mathcal{T}(\mathbf{B})]_{i,j} = -\mathbf{B}_{N-1-i, N-1-j}, \quad i, j \in \{0, \dots, N-1\}. \quad (3.1)$$

Это простое преобразование совмещает пространственную симметрию доски и замену принадлежности фигур, устраняя несущественную асимметрию между ролями игроков на уровне входных признаков.

3.3 Структура наблюдения и ограничения действий

Наблюдение агента включает две основные части:

1. числовое представление текущего состояния игрового поля;
2. маску допустимых действий.

В программной реализации наблюдение Tic-Tac-Toe задаётся словарём с двумя ключами: **observation** содержит признаки игрового поля, а **action_mask** задаёт булеву маску допустимых действий.

Для Einstein Würfelt Nicht! структура наблюдения шире: в неё входят поле, значение кубика, допустимые фигуры, фаза партии, номер расставляемой фигуры и маска действий. В обоих случаях маска допустимых действий передаётся вместе с признаками состояния и используется как ограничение при выборе действия.

3.4 Рабочий процесс обучения на базе RLlib

Низкоуровневые операции PPO, включая сбор траекторий, оценку преимуществ и градиентное обновление сети, выполняются реализацией RLlib. На уровне данной работы экспериментальный workflow состоит из следующих этапов: инициализируются main policy, фиксированные opponent strategies и history pool; один или несколько env runners собирают траектории взаимодействия со средой; перед выбором действия к logits политики применяется маска допустимых действий; policy mapping function определяет, использует ли текущий агент main policy, random policy, minimax policy или одну из исторических стратегий; центральный learner обновляет

main policy; затем с заданным интервалом выполняются оценка, сохранение checkpoint и обновление history pool.

4 Экспериментальное исследование на примере Tic-Tac-Toe

4.1 Описание игры

Tic-Tac-Toe представляет собой пошаговую игру двух игроков на поле 3×3 . Игроки по очереди ставят свои знаки в свободные клетки. Побеждает игрок, который первым построит линию из трёх своих знаков по горизонтали, вертикали или диагонали. Если поле заполнено и ни один игрок не победил, партия завершается ничьей.

Эта игра имеет малое число состояний и может быть полностью описана алгоритмически. Поэтому она используется как базовый пример для проверки корректности среды, функции награды и процедуры обучения.

4.2 Состояние и действия

В программной реализации среда передаёт состояние в виде вектора длины 9. Наблюдение строится относительно текущего игрока: значение 0 соответствует пустой клетке, значение 1 — собственной фигуре текущего игрока, значение 2 — фигуре противника. Такое кодирование позволяет использовать одну общую политику для обоих игроков.

Формально элементы наблюдения принимают значения

$$b_i \in \{0, 1, 2\}, \quad i = 1, \dots, 9, \quad (4.1)$$

где 0 обозначает пустую клетку, 1 — знак текущего игрока, а 2 — знак противника.

Кодировка 0, 1, 2 сохраняется на уровне среды и используется как компактное представление состояния, удобное для маски допустимых действий, случайного противника и minimax-противника. Однако непосредственно подавать эти значения в первый полносвязный слой нейронной сети нежелательно, поскольку это категориальные признаки, а

не числовая шкала. Значение 2 не означает, что фигура противника “в два раза больше” собственной фигуры. Поэтому внутри модели перед первым линейным слоем используется разделение каналов: исходный вектор доски преобразуется в два бинарных канала — канал собственных фигур и канал фигур противника:

$$x_i^{\text{own}} = \mathbb{I}\{b_i = 1\}, \quad x_i^{\text{opp}} = \mathbb{I}\{b_i = 2\}. \quad (4.2)$$

После объединения этих каналов вход нейронной сети имеет размерность 18:

$$x = (x_1^{\text{own}}, \dots, x_9^{\text{own}}, x_1^{\text{opp}}, \dots, x_9^{\text{opp}}) \in \{0, 1\}^{18}. \quad (4.3)$$

Такое представление устраняет ложный порядковый смысл кодов 0, 1, 2 и ближе к многоканальным представлениям доски, используемым в современных игровых системах. Для небольшой игры Тис-Тас-Тое это не является обязательным условием, но делает вход модели более корректным и обычно ускоряет формирование устойчивой стратегии.

Пространство действий содержит 9 возможных ходов, каждый из которых соответствует выбору одной клетки поля. Если клетка уже занята, соответствующее действие запрещается маской допустимых действий.

4.3 Вариант награды только за победу

В первой серии экспериментов положительная награда выдаётся только за победу. При таком подходе агент получает прямой сигнал о выигрышном исходе, однако ничья не рассматривается как полезный результат. Поэтому модель может хуже обучаться избегать поражения.

Функцию награды можно записать в виде

$$r = \begin{cases} 1, & \text{если текущий игрок победил,} \\ -1, & \text{если текущий игрок проиграл,} \\ 0, & \text{в остальных случаях.} \end{cases} \quad (4.4)$$

4.4 Вариант награды за победу и ничью

Во второй серии экспериментов учитывается не только победа, но и ничья как не проигранный исход. Такая награда лучше согласуется со свойством Tic-Tac-Toe: при оптимальной игре обеих сторон партия должна завершаться не поражением, а ничьей. Этот вариант требует отдельного запуска среды с положительной наградой за ничью.

Один из возможных вариантов функции награды имеет вид

$$r = \begin{cases} 1, & \text{если текущий игрок победил,} \\ c, & \text{если партия завершилась ничьей,} \\ -1, & \text{если текущий игрок проиграл,} \\ 0, & \text{иначе,} \end{cases} \quad (4.5)$$

где $0 < c < 1$ — награда за ничью. В численных экспериментах значение c подбирается так, чтобы поощрять стратегию, избегающую поражения.

4.5 Метрики качества

Для оценки качества используются следующие показатели:

1. доля побед, ничьих и поражений обучаемой политики;
2. доля не проигранных партий, то есть сумма доли побед и доли ничьих;
3. доля не проигранных партий отдельно при игре первым и вторым игроком;
4. худшая по роли доля не проигранных партий;
5. средняя длина партии.

Доля не проигранных партий особенно важна для Tic-Tac-Toe, поскольку при оптимальной игре обеих сторон корректная стратегия не обязана выигрывать, но не должна проигрывать. Отдельная оценка по ролям также важна: в экспериментах оказалось, что стратегия значительно устойчивее играет первым игроком, чем вторым.

4.6 Постановка эксперимента

В финальной серии для Tic-Tac-Toe исследовалось влияние награды за ничью. Остальные компоненты обучения оставались фиксированными: PPO в многоагентной постановке, общая политика для двух игроков, 18-мерное представление состояния после разделения каналов, маска допустимых действий, смешанный пул противников и несимметричная выборка ролей. Значение награды за ничью варьировалось:

$$r_{\text{draw}} \in \{0, 0.2, 0.5\}.$$

Для каждого значения r_{draw} были выполнены три независимых запуска на 500 итераций с разными seed и один дополнительный запуск на 1000 итераций. Финальная оценка каждого запуска проводилась на 1000 партиях против случайного противника и на 1000 партиях против minimax; роль обучаемой политики чередовалась между первым и вторым игроком.

Table 4.1. Параметры финальной серии экспериментов PPO для Tic-Tac-Toe.

Параметр	Значение
Алгоритм	PPO
Схема политики	shared policy для обоих игроков
Вход нейронной сети	18 признаков после разделения каналов доски
Противники при обучении	random, minimax, исторические политики
Награда за победу	1
Награда за ничью	0, 0.2, 0.5
Награда за поражение	-1
Основная серия	500 итераций, 3 независимых запуска для каждого значения r_{draw}
Дополнительная проверка	1000 итераций, 1 запуск для каждого значения r_{draw}
Размер batch	4000 шагов среды
Размер minibatch	256
Число эпох PPO на batch	10
Коэффициент обучения	10^{-4}
Коэффициент дисконтирования	0.99
Коэффициент энтропии	расписание $0.001 \rightarrow 0.0002 \rightarrow 0$
Период оценки	каждые 50 итераций
Число партий в финальной оценке	1000 против каждого противника

4.6.1 Смешанный пул противников

Обучение не ограничивается обычным self-play только против текущей версии модели. В пул противников входят три типа opponent strategies. Случайная стратегия создаёт простые тактические ошибки, на которых модель учится выигрывать партии. Minimax-стратегия служит строгим алгоритмическим ориентиром и заставляет модель учиться защитным ходам. Исторические версии обучаемой политики формируют небольшой history pool и уменьшают риск переобучения только на текущего противника или забывания ранее освоенных ситуаций.

4.6.2 Асимметричная выборка ролей

В Tic-Tac-Toe игра вторым игроком оказалась более сложной для PPO-стратегии: именно в этой роли возникали поражения против minimax. Поэтому в основной серии использовалась несимметричная схема policy mapping. В частности, около 72% тренировочных партий соответствуют ситуации, где main policy играет вторым игроком против minimax, ещё около 16% — игре main policy первым игроком против minimax. Такая настройка не меняет алгоритм PPO, но задаёт curriculum по трудным позициям и целенаправленно увеличивает долю защитных сценариев.

4.7 Результаты экспериментов

В этом разделе используются только результаты финальной серии экспериментов с единым протоколом оценки. Поэтому ранние промежуточные запуски не включаются в таблицы: все сравнения ниже выполняются при одной и той же архитектуре, одном и том же пуле противников и одинаковом числе партий в финальной оценке.

Table 4.2. Результаты 500 итераций обучения: средние значения по трём независимым запускам.

r_{draw}	Random: победы/ничьи/поражения	Random win rate	Minimax: победы/ничьи/поражения	Minimax не контроль	Minimax, игрок 2	Ср. длина против minimax
0.0	838.0/137.3/24.7	83.80%	0.0/955.7/44.3	95.57%	91.13%	8.90
0.2	784.3/184.0/31.7	78.43%	0.0/942.3/57.7	94.23%	88.47%	8.84
0.5	730.0/242.3/27.7	73.00%	0.0/952.0/48.0	95.20%	90.40%	8.88

По результатам основной серии лучшая средняя доля не проигранных партий против `minimax` была получена при $r_{\text{draw}} = 0$: 95.57% в целом и 91.13% при игре вторым игроком. При этом увеличение награды за ничью не дало монотонного улучшения. Значение $r_{\text{draw}} = 0.5$ сделало стратегию более осторожной: доля побед против случайного противника снизилась до 73.00%, а число ничьих против `random` выросло. Это показывает компромисс между защитной устойчивостью и способностью использовать ошибки слабого соперника.

Table 4.3. Дополнительная проверка на 1000 итераций обучения: один запуск для каждого значения награды за ничью.

r_{draw}	Random: победы/ничьи/поражения	Random win rate	Minimax: победы/ничьи/поражения	Minimax не контроль	Minimax, игрок 2	Ср. длина против minimax
0.0	831/136/33	83.10%	0/950/50	95.00%	90.00%	8.85
0.2	768/194/38	76.80%	0/923/77	92.30%	84.60%	8.77
0.5	770/193/37	77.00%	0/945/55	94.50%	89.00%	8.86

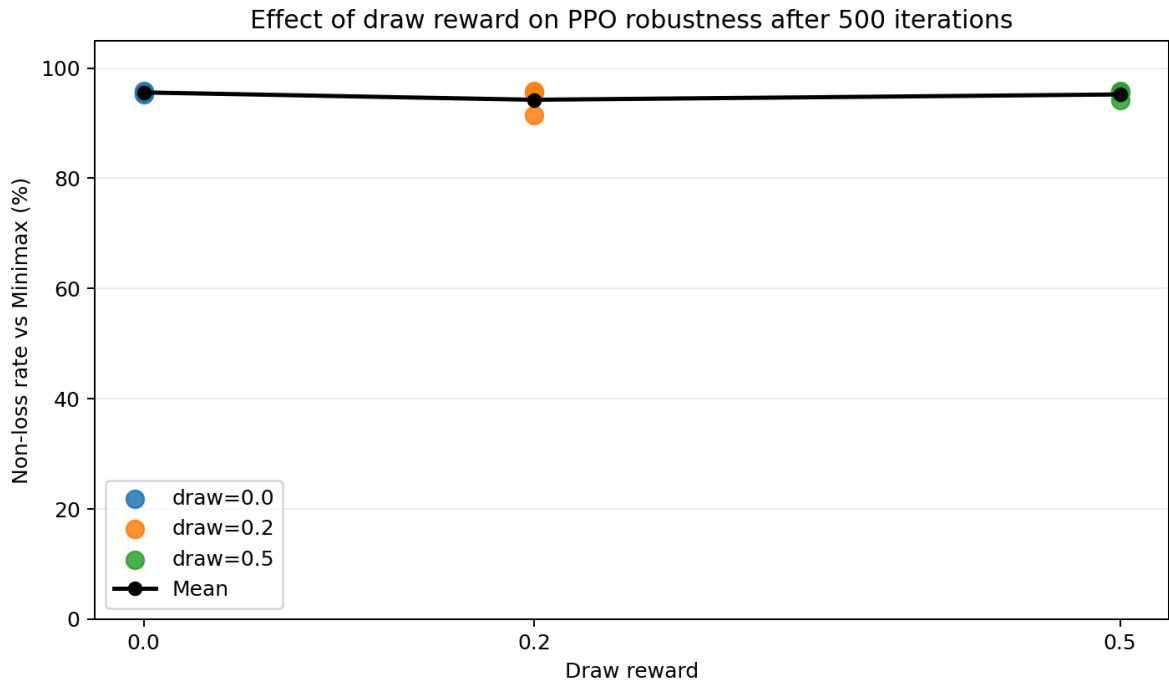


Figure 4.1. Доля не проигранных партий против `minimax` после 500 итераций для разных значений награды за ничью.

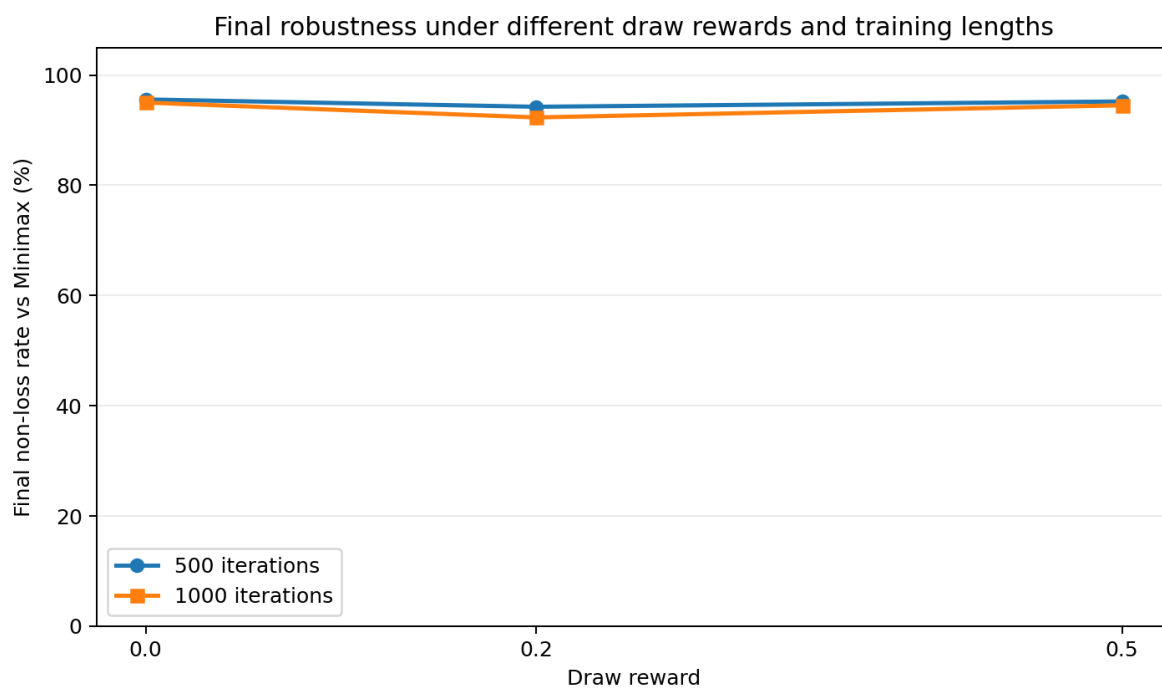


Figure 4.2. Сравнение финальной доли не проигранных партий против minimax после 500 и 1000 итераций.

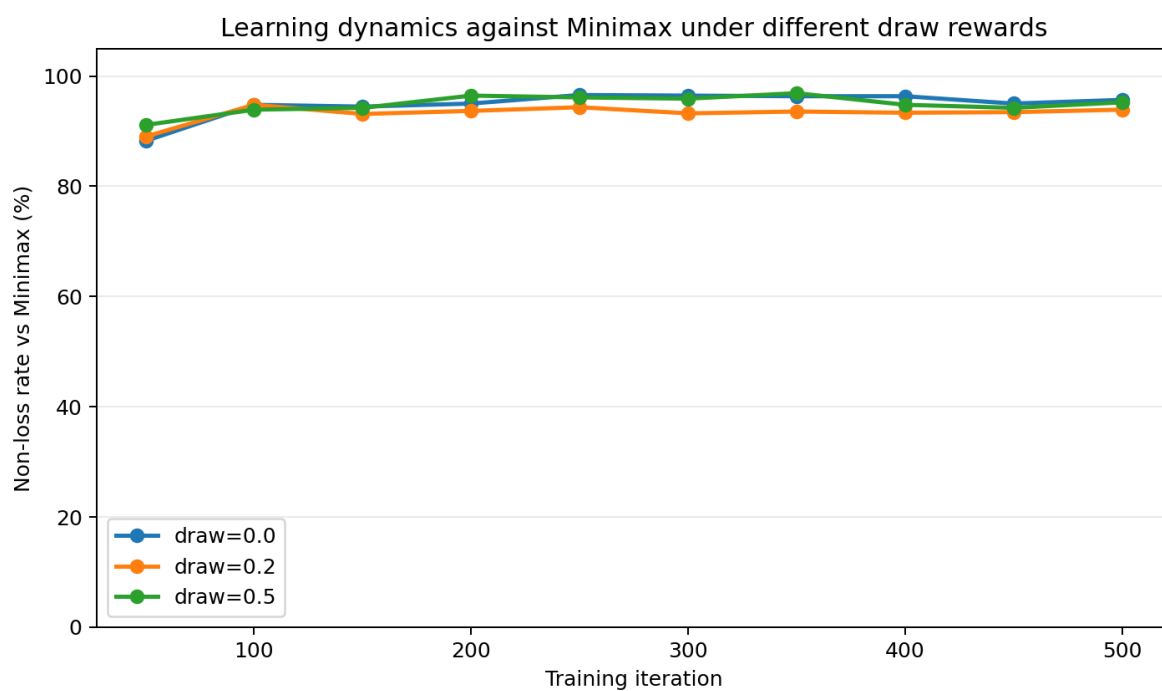


Figure 4.3. Динамика оценки против minimax в основной серии на 500 итераций.

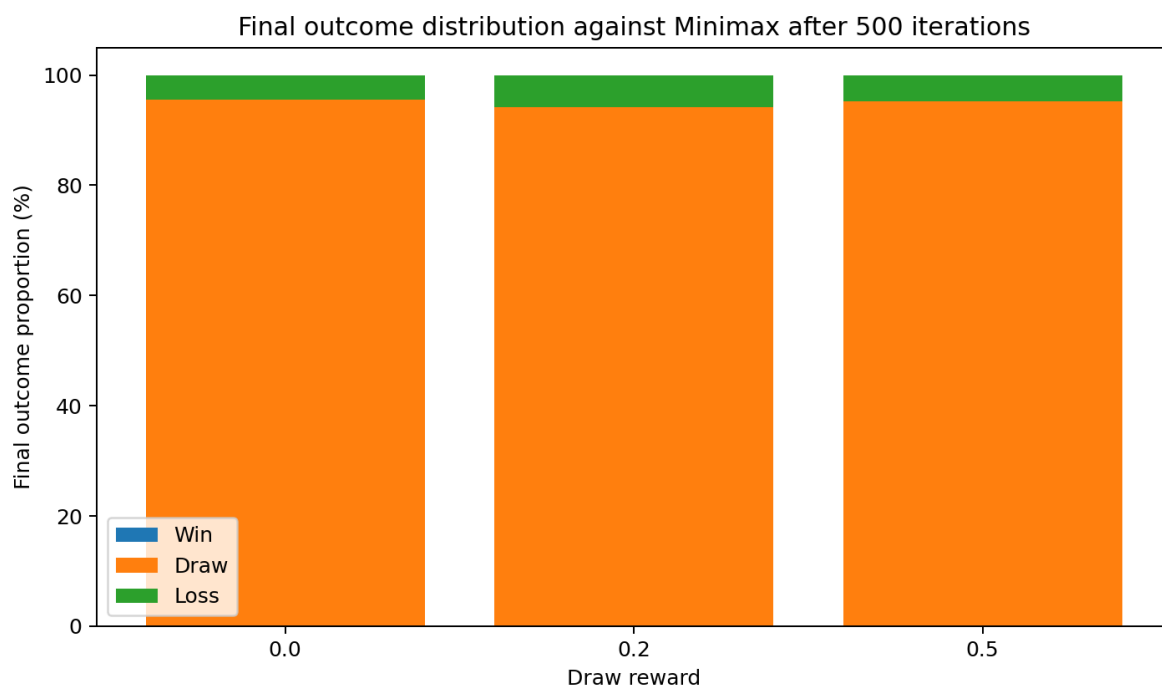


Figure 4.4. Распределение побед, ничьих и поражений против minimax после 500 итераций.

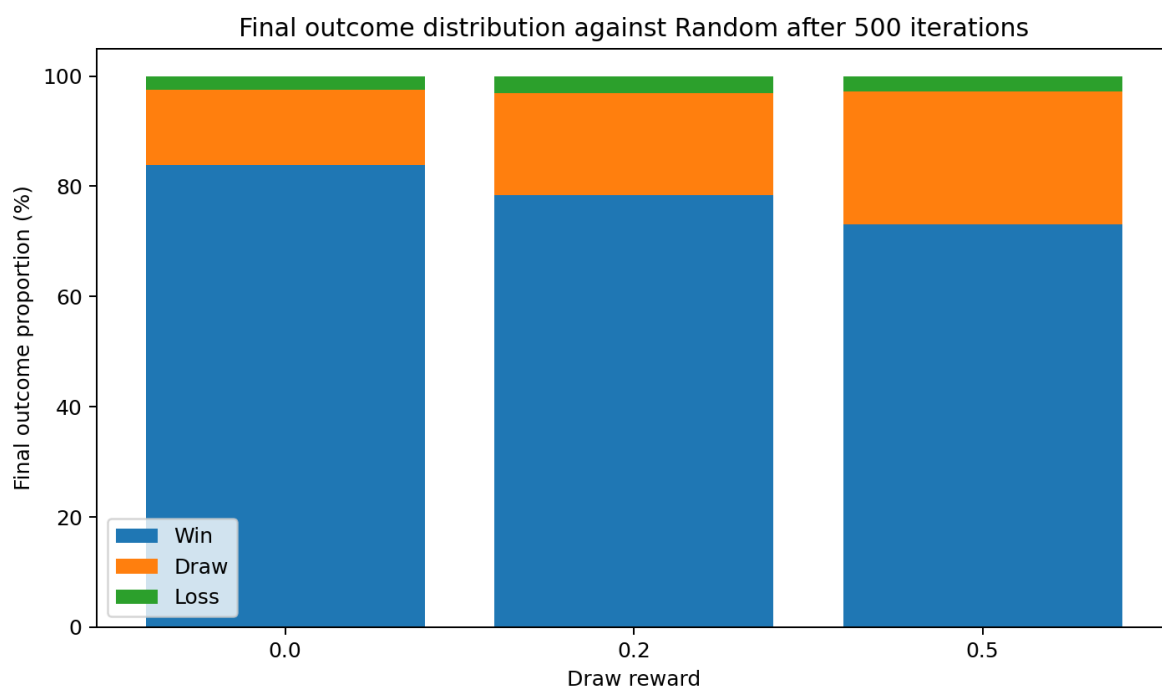


Figure 4.5. Распределение побед, ничьих и поражений против случайного противника после 500 итераций.

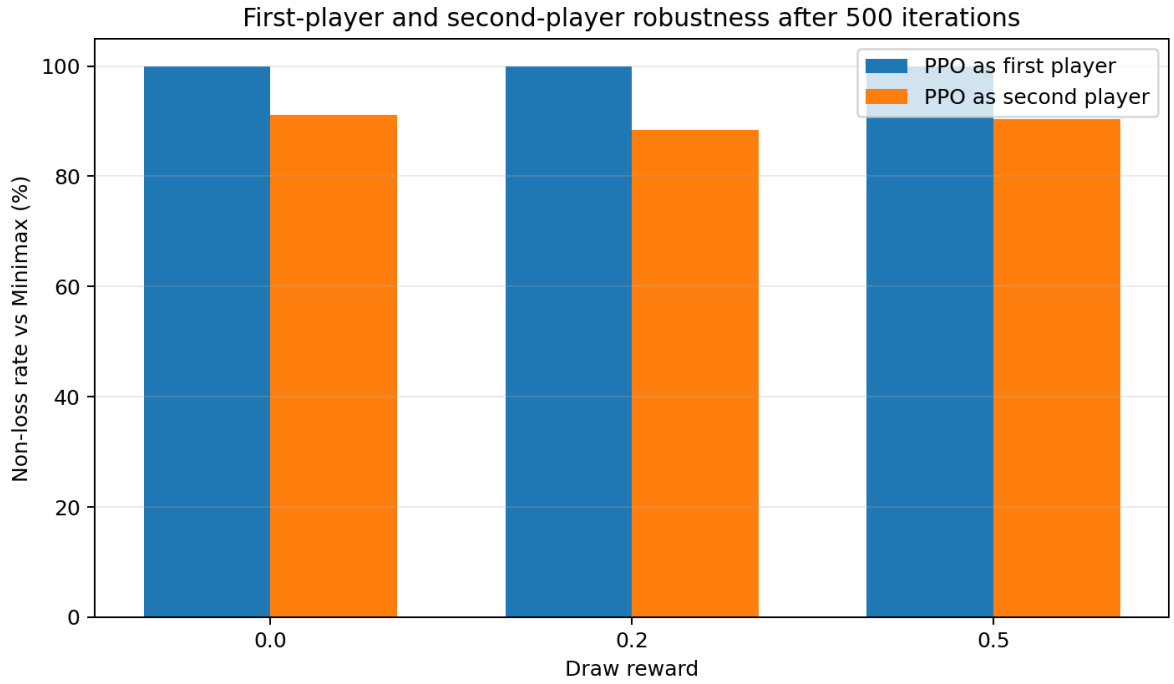


Figure 4.6. Разделение доли не проигранных партий против minimax по ролям после 500 итераций.

Поскольку основным критерием в данной работе является отсутствие поражения, сравнение по ролям проводится прежде всего по доле не проигранных партий. Для случайного противника дополнительно указывается доля побед, так как она показывает способность PPO-стратегии использовать ошибки слабого соперника. Такое сравнение приведено в таблице 4.4.

Table 4.4. Сравнение результатов по ролям после 500 итераций обучения: средние значения по трём запускам.

r_{draw}	Minimax P1 не проиграл	Minimax P2 не проиграл	Random P1 не проиграл	Random P2 не проиграл	Random P1 win rate	Random P2 win rate
0.0	100.00%	91.13%	100.00%	95.07%	92.67%	74.93%
0.2	100.00%	88.47%	100.00%	93.67%	90.87%	66.00%
0.5	100.00%	90.40%	100.00%	94.47%	84.33%	61.67%

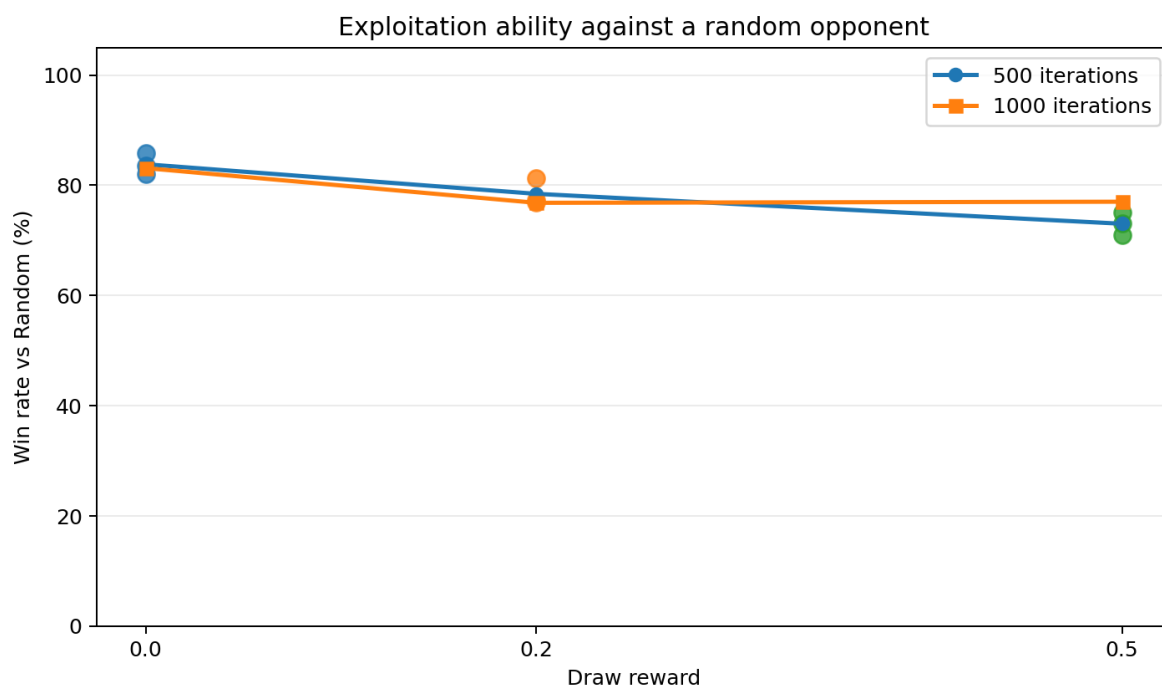


Figure 4.7. Доля побед против случайного противника для разных значений награды за ничью.

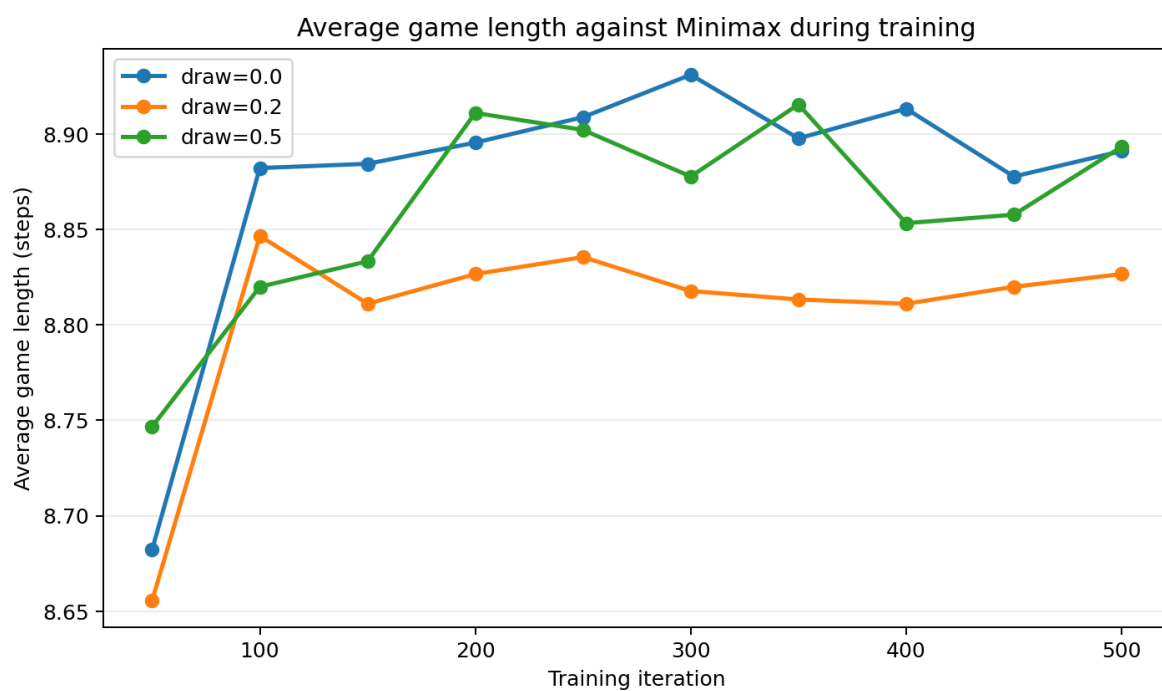


Figure 4.8. Средняя длина партии против minimax в основной серии на 500 итераций.

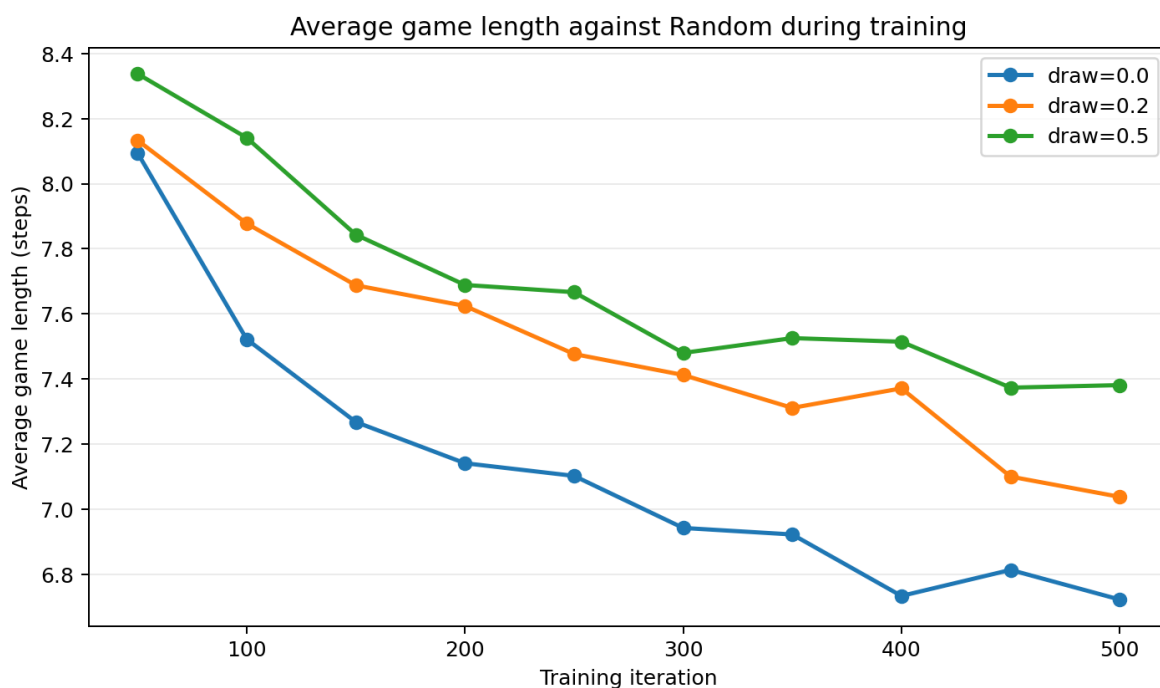


Figure 4.9. Средняя длина партии против случайного противника в основной серии на 500 итераций.

Дополнительные запуски на 1000 итераций показали, что простое увеличение числа итераций не гарантирует улучшения. Для $r_{\text{draw}} = 0$ результат против minimax снизился с 95.57% в средней оценке на 500 итерациях до 95.00% после 1000 итераций; для $r_{\text{draw}} = 0.2$ снижение оказалось более заметным, до 92.30%. Это указывает на необходимость выбирать контрольную точку по валидационной оценке, а не только по числу итераций обучения.

Диагностика проигранных партий показывает, что ошибки концентрируются в игре за второго игрока. Во всех финальных оценках первый игрок против minimax не проиграл ни одной партии, тогда как поражения возникали именно при игре вторым. Это означает, что модель иногда не выполняет обязательную защиту от немедленной угрозы, например от двойной угрозы противника, хотя такая защита легко находится точным алгоритмом.

Такое поведение связано с различием между оптимизацией ожидаемой награды и minimax -критерием наихудшего случая. РРО максимизирует средний результат на заданном распределении противников. Если в этом распределении присутствуют случайные или слабые стратегии, более активный ход может давать высокий ожидаемый выигрыш, даже если

против идеального minimax -противника он оставляет тактическую слабость. Поэтому обученная политика может иметь высокий win rate против random opponent, но всё ещё не обладать строгой гарантией не проиграть против алгоритмически оптимального соперника.

Повышение r_{draw} действительно делает стратегию менее рискованной, однако такая осторожность не эквивалентна совершенной minimax -защите. Награда за ничью является глобальным терминальным сигналом: она сообщает модели, что исход партии был приемлемым, но не указывает, какой именно локальный ход должен был заблокировать конкретную тактическую угрозу. Поэтому небольшое число ошибок в ключевых позициях может сохраняться даже при более высокой награде за ничью. Для устранения таких ошибок потребовалось бы отдельное дообучение на конкретных тактических состояниях или добавление алгоритмического фильтра безопасности ходов.

Разница между ролями также связана со структурой дерева игры Tic-Tac-Toe. Первый игрок имеет больше устойчивых вариантов развития партии и при корректной игре может как минимум удерживать ничью. Второй игрок с первых ходов вынужден отвечать на инициативу соперника: в некоторых позициях множество ходов, сохраняющих равновесный результат, значительно уже. Поэтому случайность исследования PPO и оптимизация среднего результата чаще приводят к ошибкам именно при игре вторым игроком, что объясняет необходимость несимметричной выборки ролей в обучении.

4.8 Вывод по эксперименту Tic-Tac-Toe

Эксперимент с Tic-Tac-Toe показывает, что обучение с подкреплением чувствительно к выбору функции награды, но влияние награды за ничью не является монотонным. В финальной серии лучший средний результат против minimax был получен при $r_{\text{draw}} = 0$: стратегия не проиграла 95.57% партий в целом и 91.13% партий при игре вторым игроком. Более высокая награда за ничью делала поведение более осторожным и уменьшала долю побед против случайного противника, но не устраняла отдельные ошибки в критических защитных позициях.

В то же время Tic-Tac-Toe не является задачей, где обучение

с подкреплением имеет принципиальное преимущество над точными алгоритмами. Игра имеет малое число состояний, полную информацию и может быть решена алгоритмом `minimax`. Поэтому основной вывод по данному разделу состоит в следующем: Tic-Tac-Toe полезна как проверочный пример для среды, награды и процедуры обучения, но для получения гарантированно не проигрывающей стратегии точный алгоритм надёжнее чистого RL. Дальнейшее улучшение возможно через дообучение на конкретных тактических позициях или через добавление алгоритмического фильтра ходов, однако для целей данной работы такой вариант менее интересен, поскольку главный объект исследования — более сложная игра со случайностью.

5 Обучение агента для игры Einstein Würfelt Nicht!

5.1 Правила игры

Einstein Würfelt Nicht! — пошаговая игра двух игроков на поле 5×5 . У каждого игрока есть шесть пронумерованных фигур. Выбор фигуры для хода связан со значением броска кубика: если фигура с выпавшим номером уже отсутствует, игрок может выбрать ближайшую доступную фигуру с меньшим или большим номером. В отличие от Tic-Tac-Toe, здесь присутствует случайность, поскольку на каждом ходе значение кубика влияет на множество допустимых действий.

В работе используется следующая общая логика правил:

1. перед началом игровой фазы задаётся начальная конфигурация фигур;
2. на ходе игрока определяется значение кубика;
3. по значению кубика выбираются одна или две допустимые фигуры;
4. игрок выбирает фигуру и одно из разрешённых направлений движения;

5. партия завершается при достижении целевой клетки, при уничтожении всех фигур противника, при невозможности сделать ход или при превышении максимальной длины партии.

В используемой среде первый игрок движется к правому нижнему углу, а второй игрок — к левому верхнему углу. Для удобства обучения наблюдение второго игрока поворачивается на 180° и меняет знаки фигур, поэтому одна и та же нейронная сеть может применяться к обоим игрокам.

5.2 Особенности задачи

Игра Einstein Würfelt Nicht! сложнее Tic-Tac-Toe по нескольким причинам. Во-первых, начальная конфигурация поля может изменяться перед началом партии. Во-вторых, допустимые действия зависят от случайного броска кубика. В-третьих, дерево возможных партий имеет более сложную структуру ветвления.

Эти факторы делают ручное построение универсальной стратегии более трудным и повышают интерес к методам обучения с подкреплением.

5.3 Представление состояния

Состояние среды должно содержать информацию о расположении фигур, текущем игроке, значении кубика и множестве фигур, доступных для хода. В простейшей форме наблюдение можно записать как набор признаков

$$s = (B, d, C, \varphi, q), \quad (5.1)$$

где B — закодированное поле, d — значение кубика, C — допустимые по кубику фигуры, φ — фаза партии, а q — номер фигуры в фазе начальной расстановки.

В программной реализации наблюдение состоит из следующих компонентов: поле 5×5 со значениями от -6 до 6 , значение кубика, два номера возможных фигур, маска из 6 действий, фаза партии и номер расставляемой фигуры. Также используется маска допустимых действий, которая исключает ходы, невозможные при данном положении фигур и данном значении кубика. Значение 0 в признаках кубика, кандидатов

и номера расставляемой фигуры используется как служебная категория для фазы начальной расстановки или отсутствующего кандидата, поэтому соответствующие one-hot признаки имеют размерность 7, а не 6.

5.4 Функция награды

Для игры Einstein Würfelt Nicht! используется не только терминальная награда за победу или поражение, но и промежуточная reward shaping схема. Это необходимо из-за более длинных партий и большего числа состояний: при чисто терминальной награде сигнал обучения становится слишком разреженным.

Table 5.1. Функция награды в среде Einstein Würfelt Nicht!.

Компонент награды	Значение и смысл
Победа	+1.0 победителю
Поражение	−1.0 проигравшему
Штраф за ход	−0.01 активному игроку за каждый ход
Прогресс к цели	$0.08 \cdot \Delta d$, где Δd — уменьшение расстояния Чебышёва до целевой клетки
Взятие фигуры противника	+0.03 активному игроку и −0.03 противнику
Самовзятие	0.0 дополнительного штрафа
Нелегальный ход	−1.0 активному игроку; при включённом режиме illegal-move-loss партия завершается победой противника

Прогресс-награда поощряет движение к целевой клетке даже до завершения партии. Штраф за ход делает более короткие выигрышные траектории предпочтительными, а небольшая награда за взятие фигур помогает модели учитывать не только движение к цели, но и взаимодействие с фигурами противника.

5.5 Пространство действий

Пространство действий в реализации имеет размер 6. Во время игровой фазы действие декодируется как пара

$$a = 3c + d, \tag{5.2}$$

где $c \in \{0, 1\}$ — номер кандидата среди фигур, разрешённых броском кубика, а $d \in \{0, 1, 2\}$ — направление движения. Для первого игрока разрешённые направления соответствуют движению вправо, вниз и по диагонали вправо-вниз; для второго игрока — симметричным направлениям в сторону его целевой клетки.

В фазе начальной расстановки те же 6 действий обозначают выбор одной из шести стартовых клеток для текущей фигуры. В конкретном состоянии доступно только подмножество действий $\mathcal{A}_{\text{legal}}(s)$, определяемое правилами игры и маской допустимых ходов.

5.6 Модель и архитектура

В качестве обучаемой стратегии используется нейронная сеть, которая получает на вход признаки состояния и маску допустимых действий. Поле кодируется не одним числовым слоем, а набором бинарных каналов: пустая клетка, шесть каналов собственных фигур и шесть каналов фигур противника. Для поля 5×5 это даёт $25 \cdot 13 = 325$ признаков.

Дополнительно используются one-hot признаки значения кубика, допустимых фигур, фазы партии и номера фигуры в фазе расстановки. В текущей реализации размерность равна 355: 7 признаков для кубика соответствуют значениям $0, \dots, 6$, где 0 используется в фазе расстановки; два кандидата также кодируются как два one-hot вектора длины 7; номер расставляемой фигуры и фаза партии остаются отдельными признаками. Общая размерность входного вектора равна

$$25 \cdot 13 + 7 + 2 \cdot 7 + 7 + 2 = 355. \quad (5.3)$$

Архитектура модели имеет следующий вид:

1. входной слой принимает 355 признаков состояния;
2. три полносвязных скрытых слоя имеют размерности 256, 256 и 128;
3. в скрытых слоях используется функция активации ReLU;
4. policy-head формирует 6 логитов действий;
5. value-head формирует оценку ценности состояния;

6. маска допустимых действий применяется к логитам перед выбором хода.

5.7 Алгоритм обучения

Обучение проводится методом PPO в многоагентной постановке. Основная обучаемая политика используется как за первого, так и за второго игрока, что соответствует shared-policy подходу. В качестве противников во время обучения применяются случайная стратегия допустимых ходов и небольшой пул исторических версий обучаемой политики. Такой пул снижает риск переобучения только на текущую версию соперника.

Алгоритм 1 Обучение агента для Einstein Würfelt Nicht!

Вход: среда Einstein, начальная политика π_θ , число итераций N

Выход: обученная политика π_θ

- 1: **for** $i = 1, 2, \dots, N$ **do**
 - 2: сгенерировать партии с текущей политикой
 - 3: для каждого состояния применить маску допустимых действий
 - 4: вычислить награды за исходы партий и промежуточные события
 - 5: обновить параметры политики методом PPO
 - 6: периодически оценить модель против базового противника
 - 7: периодически сохранить текущую политику в исторический пул
 - 8: **end for**
 - 9: **return** обученная политика
-

5.8 Данные обучения и вычислительные условия

В этом подразделе фиксируются параметры обучения, использованные в эксперименте. Модель обучалась на CPU; GPU в данном запуске не использовался, поскольку сеть сравнительно мала, а существенная часть времени уходит на пошаговое выполнение среды и сбор эпизодов.

Table 5.2. Параметры обучения для игры Einstein Würfelt Nicht!.

Параметр	Значение
Алгоритм	PPO
Число итераций обучения	1000
Общее число шагов среды	4 000 000
Оценочное число завершённых обучающих эпизодов	около $2.22 \cdot 10^5$ в завершённом 1000-итерационном запуске
Размер batch	4000
Размер minibatch	256
Число эпох PPO на batch	10
Коэффициент обучения	10^{-4}
Коэффициент дисконтирования	0.99
Коэффициент энтропии	0.01
Максимальная длина партии	200 ходов
Число env runners	2
Время обучения на CPU	около 2142 с, то есть 35.7 мин
Время обучения на GPU	отдельный запуск не проводился
Используемая видеокарта	не использовалась, num_gpus=0
Версия Python	3.11.14
Версии библиотек	Ray 2.52.1, PyTorch 2.5.1, Gymnasium 1.1.1

Текущая конфигурация обучения использует CPU, поскольку параметр `num_gpus` равен 0. Отдельный GPU-запуск для этой конфигурации не выполнялся.

Table 5.3. Сравнение времени обучения на CPU и GPU.

Устройство	Число итераций	Время обучения	Относительное ускорение
CPU	1000	2142 с	1.0
GPU	не проводился	—	—

5.9 Результаты против базового алгоритмического противника

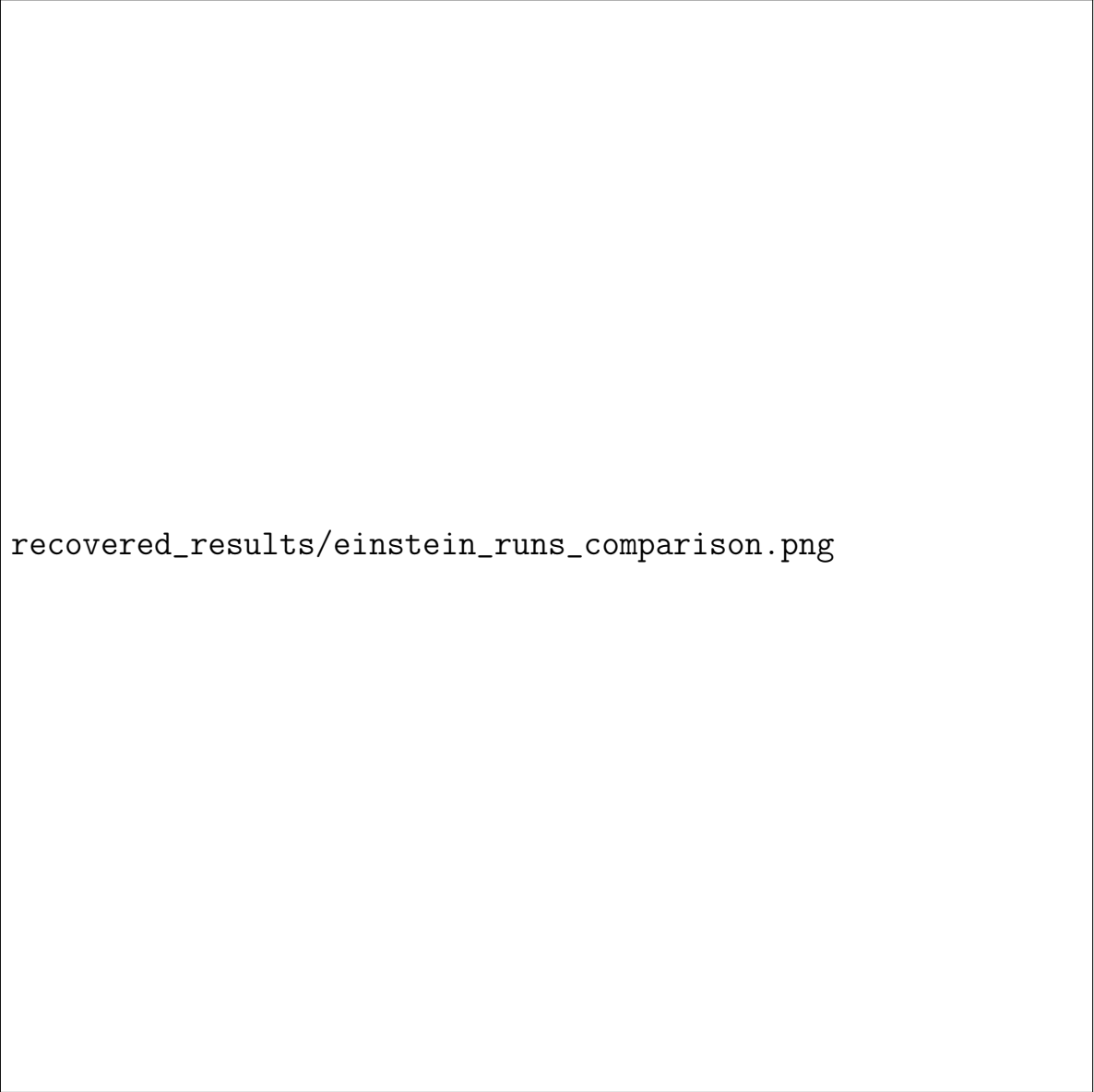
Для оценки качества обученная стратегия сравнивается с базовым алгоритмическим противником. В текущей реализации таким базовым противником является стратегия случайного выбора одного из допустимых

действий. Это простой, но воспроизводимый ориентир: он всегда соблюдает правила игры и позволяет проверить, научилась ли модель использовать структуру состояния лучше случайного игрока.

В текущей серии была проведена оценка сохранённой контрольной точки `checkpoints_einstein_setup` на 1000 партиях против случайного допустимого хода. Роль модели чередовалась: в половине партий она играла первым игроком, в половине — вторым.

Table 5.4. Результаты обученного агента против базовых противников.

Противник	Число партий	Победы агента	Поражения агента	Средняя длина партии
Случайный допустимый ход	1000	758	242	18.88
Правиловой противник	не реализован	—	—	—
Алгоритмический противник	не реализован	—	—	—



recovered_results/einstein_runs_comparison.png

Figure 5.1. Сравнение динамики нескольких запусков обучения для игры Einstein Würfelt Nicht!.

5.10 Анализ результатов

Результаты необходимо интерпретировать с учётом сложности игры. Если для Tic-Tac-Toe точный алгоритм может полностью описать оптимальную стратегию, то для Einstein Würfelt Nicht! ситуация сложнее: начальная конфигурация и случайный бросок кубика увеличивают разнообразие состояний. Поэтому здесь методы обучения с подкреплением могут быть более полезны как способ автоматического построения стратегии на основе большого числа сыгранных партий.

В оценке против случайного допустимого хода обученная стратегия выиграла 758 партий из 1000, то есть показала win rate 75.8%. При

игре первым модель выиграла 79.4% партий, при игре вторым — 72.2%. Все партии завершились без ничьих или усечений по лимиту ходов: 996 партий закончились достижением целевой клетки, ещё 4 — уничтожением всех фигур соперника. Это показывает, что агент научился использовать структуру игры лучше случайного противника, хотя разница между первым и вторым игроком сохраняется.

6 Заключение

В работе рассмотрено применение методов обучения с подкреплением к пошаговым играм. На примере Tic-Tac-Toe показано, что для простой полностью определённой игры обучение с подкреплением позволяет получить работоспособную стратегию, однако такая игра может быть решена точными алгоритмическими методами. Поэтому для Tic-Tac-Toe методы обучения с подкреплением имеют прежде всего демонстрационное и проверочное значение.

Эксперименты с различными функциями награды показали, что качество обученной стратегии существенно зависит от того, как задана награда. Если награда выдаётся только за победу, агент может недостаточно хорошо избегать поражения. Если же учитывается также ничья, то стратегия становится более устойчивой по критерию не проигранных партий.

Для игры Einstein Würfelt Nicht! построена более сложная среда, учитывающая изменяемую начальную конфигурацию, случайный бросок кубика и маску допустимых действий. Такая постановка лучше демонстрирует область, где обучение с подкреплением может быть полезным: большое число возможных состояний, случайность и сложное дерево решений затрудняют ручное построение универсальной стратегии.

Основной общий вывод состоит в следующем: для простых детерминированных игр с малым числом состояний точные алгоритмы обычно оказываются более надёжными и эффективными. Однако для игр с большим числом состояний, случайными факторами и сложной структурой ветвления методы обучения с подкреплением являются перспективным подходом, поскольку позволяют обучать стратегию непосредственно на

опыте взаимодействия со средой.

Дальнейшее развитие работы может быть связано с улучшением функции награды, использованием curriculum learning, сравнением с более сильными алгоритмическими противниками, анализом неудачных траекторий и расширением архитектуры модели.

References

- [1] Sutton R. S., Barto A. G. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.
- [2] Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
- [3] Mnih V., Kavukcuoglu K., Silver D. et al. Human-level control through deep reinforcement learning. *Nature*, 2015, Vol. 518, pp. 529–533.
- [4] Silver D., Huang A., Maddison C. J. et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 2016, Vol. 529, pp. 484–489.
- [5] Liang E., Liaw R., Nishihara R. et al. RLlib: Abstractions for Distributed Reinforcement Learning. *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [6] Shoham Y., Powers R., Grenager T. Multi-agent reinforcement learning: a critical survey. Technical Report, Stanford University, 2008.
- [7] Russell S., Norvig P. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.